

AGENTIC AI DRIVEN MULTI AGENT SYSTEM FOR INTELLIGENT TRAFFIC MANAGEMENT

The Complete Visual Manual, Architecture Tree, Codebase Walkthrough & Dashboard Guide



Figure 1: 3D Visual Rendering of the AI-Controlled 4-Way Smart Intersection with Emergency Green Wave Preemption

■ **The Mission:** In modern smart cities, fixed timer traffic lights are the root cause of traffic gridlock, wasted fuel, and emergency delays. Our project builds an autonomous cyber-physical system powered by **4 collaborative AI agents**, connected to **physical ESP32 microcontrollers over MQTT**, simulated at high fidelity in **Eclipse SUMO**, and monitored in real-time on a **60 FPS React Digital Twin Dashboard**. This master document explains every diagram, code module, tool, and dashboard panel in complete detail!

1. Codebase Directory Tree & System Structure

Our project workspace in `c:\Traffic Project\` is cleanly partitioned into three specialized sub-projects: **BACKEND (Python)**, **FRONTEND (JavaScript/React)**, and **FIRMWARE (C++ Arduino)**:

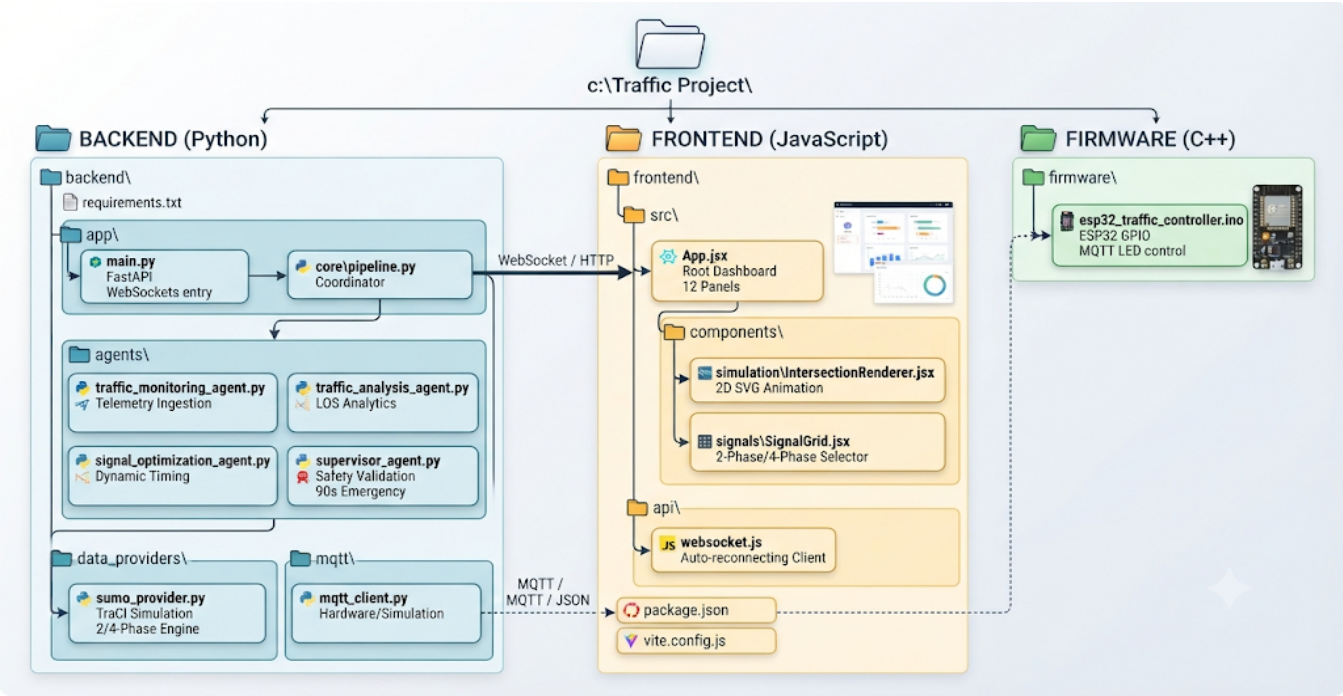


Figure 2: Architectural Tree of the 'Traffic Project' Folder connecting Backend, Frontend, and ESP32 Firmware

- **BACKEND (Python):** Runs the FastAPI web server, WebSocket dispatchers, the 4-Agent pipeline loop, and SUMO TraCI micro-simulator.
- **FRONTEND (JavaScript):** Houses the React 18 / Vite 5 dashboard, 2D SVG canvas simulation, and interactive mode toggles.
- **FIRMWARE (C++):** Embedded code running on ESP32 Dev Module driving physical GPIO pins to control RED/YELLOW/GREEN LED hardware.

2. Plain-English Guide to Every Technology We Used

Here is what every tool, language, and framework does in our project, explained with intuitive everyday analogies:

Technology	Analogy	What It Does & Why We Used It
Python 3.10+	The Brain	Executes the 4-agent decision logic, calculates green times, and processes live SUMO TraCI vehicle telemetry arrays.
FastAPI & Uvicorn	The Speed Waiter	FastAPI handles REST API endpoints and WebSockets. Uvicorn is the high-performance async engine serving requests on port 8001.
JSON	The Universal Note	Standardized text format with key-value pairs so Python, React, and ESP32 microcontrollers communicate without errors.
WebSockets (lws)	The Live Radio Line	A persistent full-duplex connection that streams telemetry updates to the browser at 60 FPS without page reloads.
MQTT (paho-mqtt)	The Wireless Beacon	Lightweight IoT protocol broadcasting commands to physical ESP32 nodes in under 15ms latency.
ESP32 & C++ Arduino	The Physical Hands	A dual-core 240MHz microcontroller driving GPIO pins (18, 19, 21, 22, 25, 26, 32, 33) to light real LED signal heads.
Eclipse SUMO & TraCI	The Flight Simulator	Microscopic traffic simulator modeling vehicle queues, braking distances, lane changes, and speeds at Junction J1.
React 18 & HTML5	The LEGO UI	HTML creates structure; React creates modular, reactive components (HUD, Signal Cards, Charts) that update in real time.
Vite 5 (vite.js)	The Rocket Engine	Next-gen frontend build tool with instant Hot Module Replacement (HMR) and proxy routing to the backend.

TailwindCSS & PostCSS	The Designer & Tailor	Tailwind provides sleek dark-mode glassmorphism and neon styles. PostCSS compiles and optimizes CSS rules for all browsers.
Zustand & React Query	The Memory Notebook	Zustand stores local UI states (active scenarios), and React Query caches server data with automatic background refetching.
Recharts & Framer Motion	The Magic Visuals	Recharts renders time-series area charts. Framer Motion provides smooth spring physics transitions and animations.

3. Complete Dashboard Features & Visual Representation

Below is a panel-by-panel breakdown of the 12 dashboard features running live on <http://localhost:3000>:

3.1. System Overview KPIs & Interactive 2D Junction Simulator

- **Real-Time KPI Cards:** Displays Total Vehicle Count, Average Speed (km/h), Queue Length (m), Throughput (veh/hr), and Level of Service (LOS).
- **2D SVG Canvas Digital Twin:** Animates real vehicle movement along 4 approaches with stop bar stopping physics and glowing signal heads.
- **Simulation HUD:** Shows current active phase, live countdown timer, and the active mode chip (**4-PHASE (1-BY-1)** vs **2-PHASE (PAIRED)**).

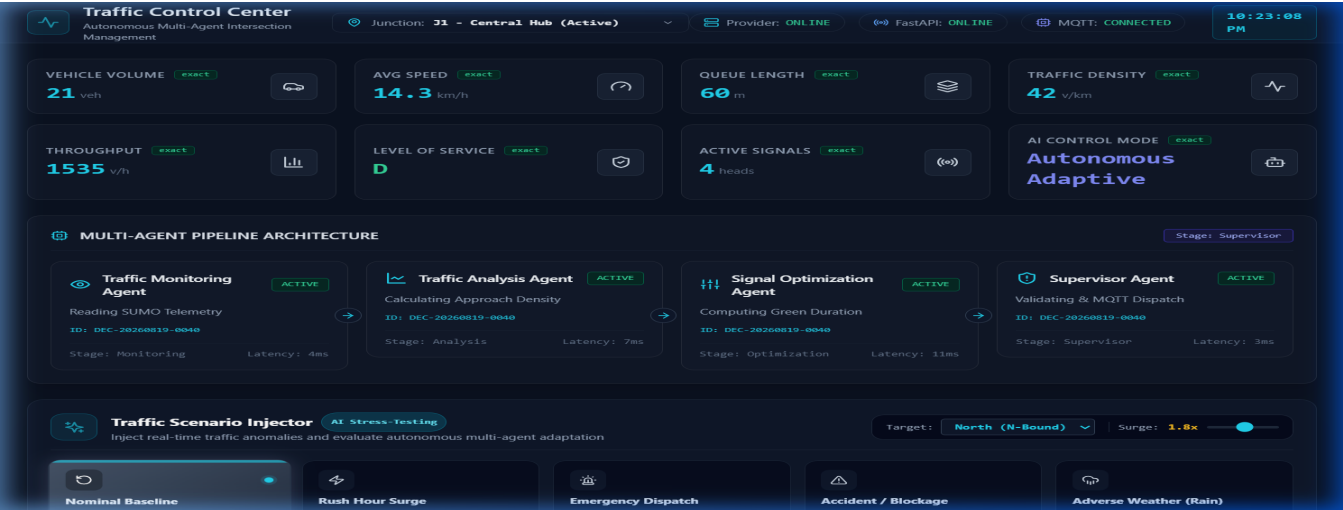


Figure 3: Real-Time Overview KPIs, 2D Digital Twin Intersection Simulator, and Active Signal Heads

3.2. Multi-Agent AI Pipeline Flow Visualizer & Approach Analytics

- **Live Multi-Agent Pipeline:** Visualizes the 4 agents with step-by-step progress chips and live execution latencies (~4ms to 12ms).
- **Per-Approach Direction Cards:** Dedicated cards for North, South, East, and West displaying vehicle counts, queue lengths, density progress bars, and congestion status badges.

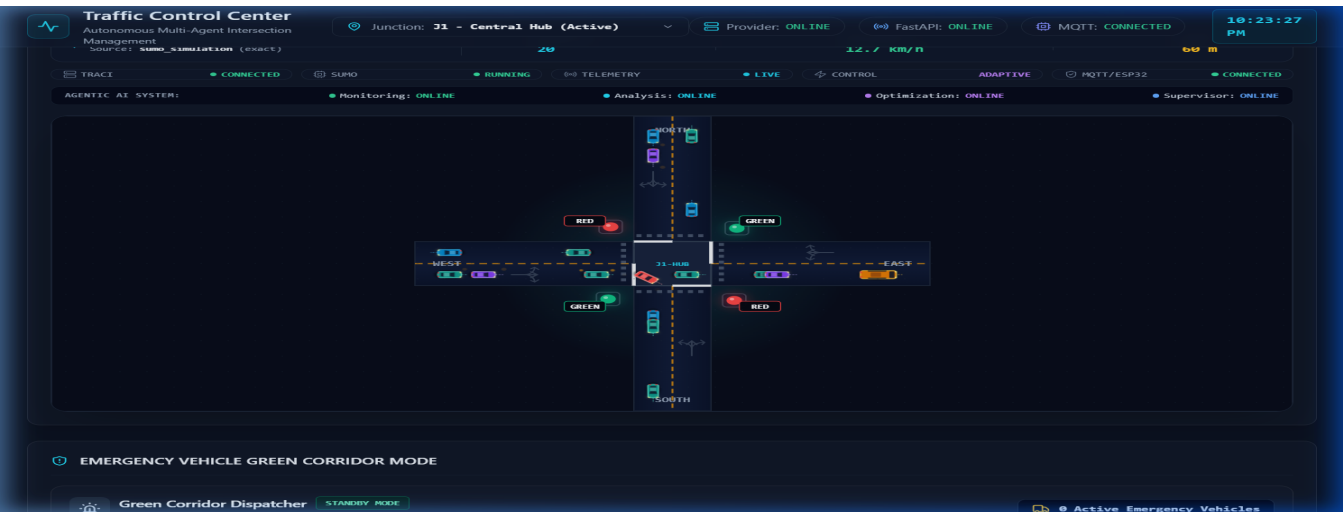


Figure 4: Multi-Agent AI Execution Flow Visualizer and Directional Approach Telemetry Cards

3.3. Emergency Green Wave Corridor & Scenario Stress-Testing Injector

- **Emergency Green Wave Override:** Instantly preempts signals to grant a continuous 90s GREEN wave for arriving ambulances or fire engines while locking opposing lanes RED.
- **Interactive Scenario Injector:** One-click buttons to inject **Rush Hour Surge**, **Accident Blockage**, **Adverse Weather (Rain/Fog)**, or return to **Normal Baseline**.

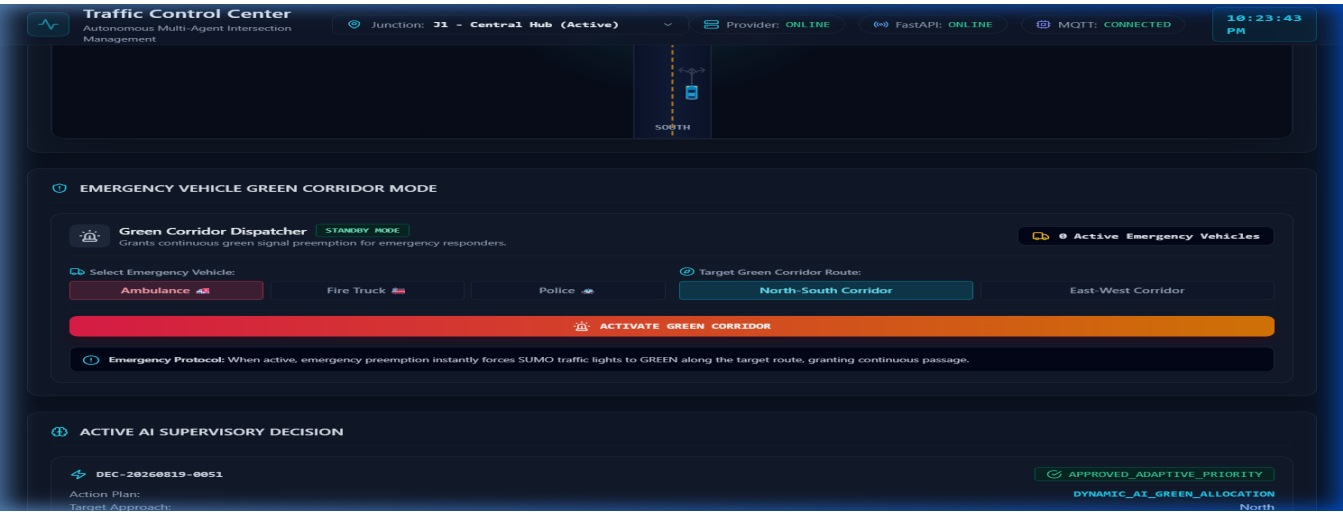


Figure 5: Emergency Green Wave Preemption Controls and Dynamic Scenario Stress-Testing Injector Panel

3.4. ESP32 Hardware IoT Monitors & Historical Recharts Trends

- **ESP32 Microcontroller Node Monitors:** Displays live ping (ms), WiFi RSSI, active firmware version, and real-time GPIO LED states (Pins 18, 19, 21, 22, 25, 26, 32, 33).
- **Time-Series Recharts Trends:** Dual responsive area charts graphing vehicle volume and average corridor velocity over rolling 15-minute, 30-minute, or 1-hour windows.

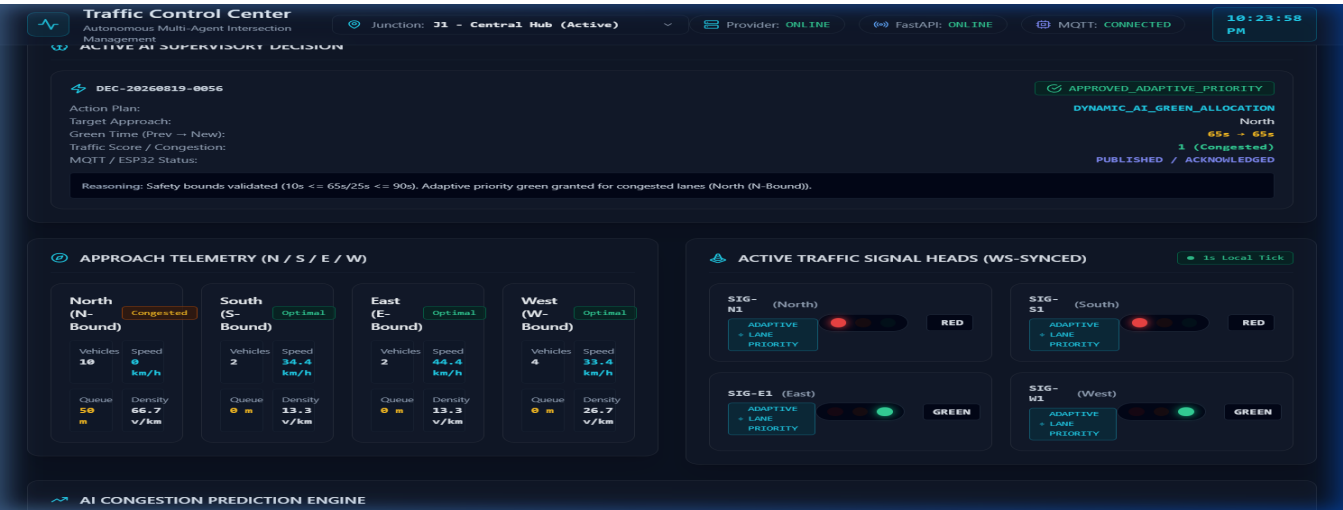


Figure 6: Physical ESP32 Hardware Node Monitor Grid and Time-Series Speed / Volume Recharts Graphs

3.5. AI Predictive Forecast Analytics (+15m, +30m, +60m) & Live TraCI Dataset Table

- **Machine Learning Predictive Analytics:** Forecasts junction congestion percentage and predicted vehicle volume at +15m, +30m, and +60m horizons using rolling regression models.
- **Live 20-Row TraCI Dataset Table:** Interactive tabular data view showing raw vehicle IDs, lane positions, velocities, and halting durations extracted directly from Eclipse SUMO.

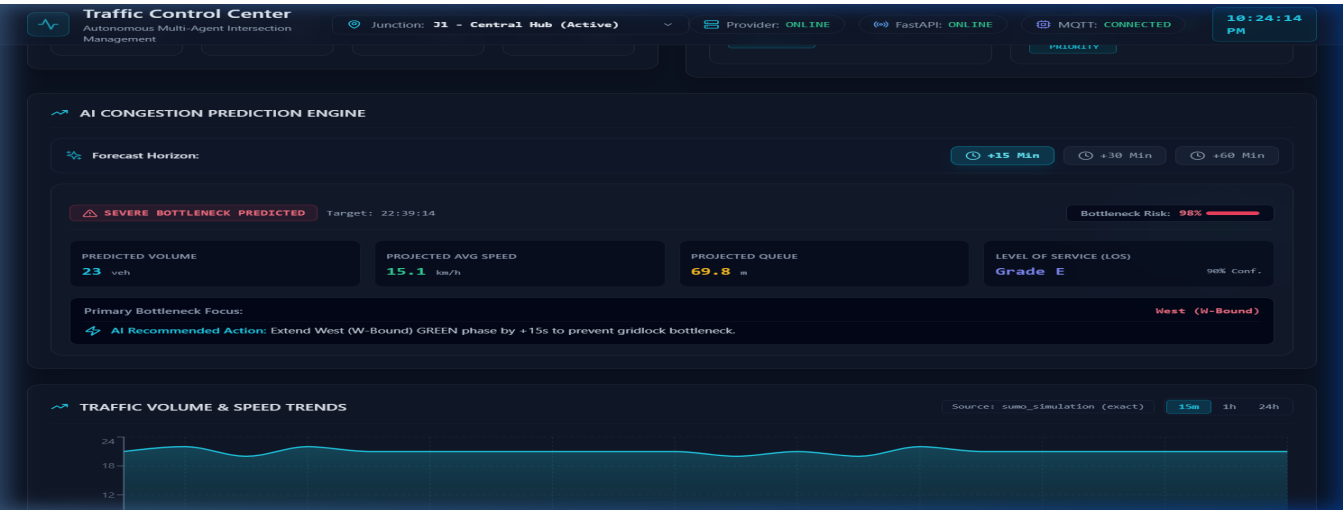


Figure 7: AI Predictive Horizon Forecasts and Live 20-Row TraCI Simulation Dataset Table

3.6. Real-Time System Activity Logs & Audit Trail

- **Live Log Stream:** Auto-scrolling terminal window displaying timestamped decision logs, MQTT publish confirmations, safety approvals, and WebSocket connection health.

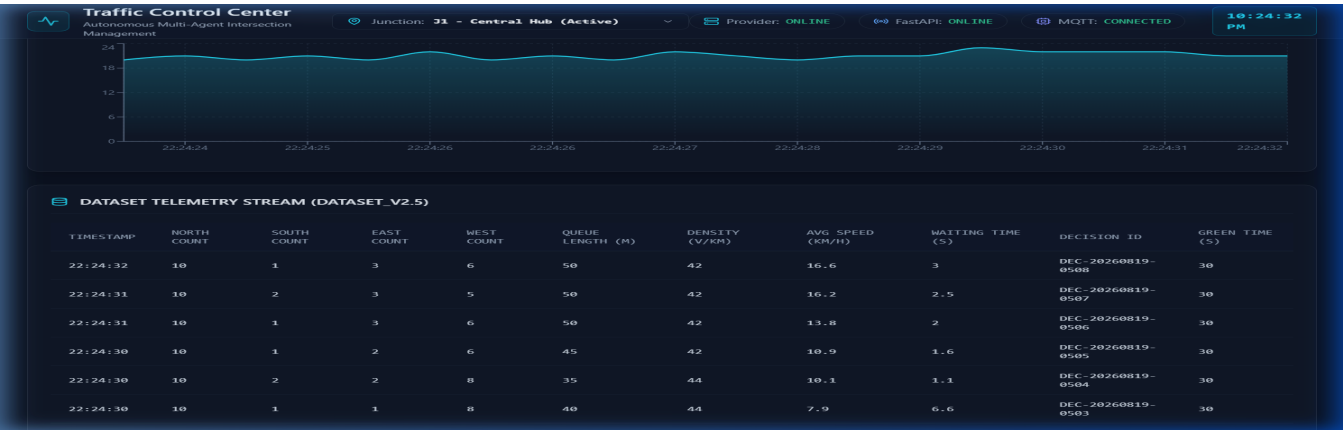


Figure 8: Real-Time System Activity Log Stream and Audit Trail Console

4. The 4 Autonomous AI Agents & Mathematical Green Model

The intelligence of our system is divided across 4 specialized agents executing sequentially every 3 seconds:

Agent 1: Sentinel (traffic_monitoring_agent.py) <i>Ingestion & Perception</i> Reads SUMO TraCI and camera feeds. Normalizes vehicle counts, approach speeds, queue lengths in meters, and emergency siren flags (~5ms latency).	Agent 2: Detective (traffic_analysis_agent.py) <i>Analytics & Pattern Recognition</i> Calculates vehicle density per km, determines bottleneck lanes, and computes junction Level of Service (LOS A to F).
Agent 3: Mastermind (signal_optimization_agent.py) <i>Dynamic Timing Plan Strategist</i> Calculates dynamic green allocations (15\text{s} \le \text{Green} \le 75\text{s}) using our multi-factor scoring formula with congestion priority boosts.	Agent 4: Commander (supervisor_agent.py) <i>Safety Guardian & Hardware Dispatcher</i> Validates safety boundaries (10\text{s} \le \text{Green} \le 90\text{s}), triggers instant 90s Emergency Green Waves, and dispatches JSON commands to MQTT.

The Dynamic Green Timing Mathematical Formula:

Base_Score = 0.35*(Density/60) + 0.30*(Queue/50) + 0.20*(Wait/60) + 0.15*(Volume/40) Final_Score = min(1.0, Base_Score + (0.25 if is_congested else 0.0)) Allocated_Green = max(15, min(75, int(15 + 50 * Final_Score)))

5. Dual Signal Sequencing Modes: 2-Phase Paired vs. 4-Phase Isolated

Operators and AI agents can switch dynamically between two signal control strategies via the UI:

Control Mode	Phase Progression	Engineering Advantages & Best Use Cases
Paired Corridors (2-Phase Mode)	Phase 1: North + South GREEN Phase 2: East + West GREEN	High Arterial Throughput: Discharges opposing traffic simultaneously. Best for wide straight avenues with heavy straight flow.
One-by-One Approach (4-Phase Isolated)	Phase 1: North GREEN only Phase 2: East GREEN only Phase 3: South GREEN only Phase 4: West GREEN only	Zero Turn Conflicts: Completely eliminates left/right turning collision hazards. Only 1 approach moves at a time while all 3 others are RED.

6. How to Run & Showcase This Project with Confidence

Terminal 1 — Start the Backend (FastAPI + Uvicorn):

```
cd "c:\Traffic Project\traffic-control-center\backend" .\venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8001 --reload
```

Terminal 2 — Start the Frontend (Vite + React):

```
cd "c:\Traffic Project\traffic-control-center\frontend" npm run dev
```

Browser Access: Open <http://localhost:3000> (Swagger API Docs at <http://localhost:8001/docs>).